

1. 题目深度解构

题意重述

给定一个可重集合 S 和目标整数 k 。

一次操作为：

选择 S 的一个子多重集 S' ，可以为空，将 S' 从 S 中删去，然后把

$$\text{mex}(S')$$

加入 S 。

目标是用最少操作次数把整个可重集合变成单元素集合：

$$S = \{k\}$$

答案对 998244353 取模。

约束分析

数据范围：

$$t \leq 10^5, \quad \sum n \leq 5 \cdot 10^5, \quad k, s_i \leq 10^9$$

关键点：

1. k 很大，不能做 $O(k)$ 。
2. 答案可能接近 2^k ，所以需要取模。
3. $\sum n$ 可控，允许对每组数据排序，总复杂度可以是：

$$O(n \log n + \log k)$$

或者整体：

$$O\left(\sum n \log n + \sum \log k\right)$$

题目类型判定

这是一道 **Ad-hoc / 思维题 + 贪心 + 逆向建模 + 稀疏递推**。

核心不是常规数据结构，而是看穿 mex 操作在逆向下的结构。

2. 思维路径推导

从朴素思路开始

如果从正向看，操作是：

$$S' \rightarrow \text{mex}(S')$$

为了得到 k ，最后一步之前，集合里必须至少有：

$$0, 1, \dots, k-1$$

并且不能有 k 。

如果从空集合构造 $\{k\}$ ，设代价为 $f(k)$ ，那么：

$$f(0) = 1$$

因为可以选择空集：

$$\emptyset \rightarrow 0$$

而要得到 k ，需要先得到所有 $0, 1, \dots, k-1$ ，最后再合成 k ，所以：

$$f(k) = 1 + \sum_{i=0}^{k-1} f(i)$$

于是：

$$f(k) = 2^k$$

这解释了为什么答案可能非常大。

但是题目给了一些初始元素，它们可以减少构造成本。

一个错误但有启发的想法

如果 S 里有一个 x ，它似乎可以替代构造 x 的代价 2^x 。

比如：

$$S = \{20, 2, 6\}, \quad k = 52$$

答案确实像：

$$2^{52} - 2^{20} - 2^6 - 2^2$$

但这个想法不完整。

反例：

$$S = \{0, 0\}, \quad k = 2$$

如果只按“不同元素”扣除，只能扣一个 0，得到：

$$2^2 - 1 = 3$$

但实际上：

$$\{0, 0\} \rightarrow \{0, 1\} \rightarrow \{2\}$$

只需要 2 次。

说明：重复元素也可能有用。

再看：

$$S = \{0, 2, 2\}, \quad k = 3$$

两个 2 并不能都发挥 2^2 的作用，因为构造 3 的结构里只需要一个 2。

所以不能简单按权值求和。

关键观察：逆向操作

正向操作：

$$S' \rightarrow \text{mex}(S')$$

逆向看，就是：

选择一个元素 x ，把它替换成任意一个 mex 为 x 的可重集合。

也就是说，逆向中：

一个 x 可以展开成一个集合 T ，满足：

1. T 必须包含 $0, 1, \dots, x-1$;
2. T 不能包含 x ;
3. 其它数可以随便放。

因此，逆向从 $\{k\}$ 出发，要生成初始集合 S 。

如果初始集合本来就是 $\{k\}$ ，答案为 0。

否则，根节点 k 必须展开一次。

展开 k 时，会强制产生：

$$0, 1, \dots, k-1$$

此外，所有不等于 k 的初始元素都可以作为“额外元素”免费塞进去。

真正麻烦的是：强制产生出来的 $0, 1, \dots, k-1$ ，如果最终不需要，就必须继续展开消掉；如果最终有对应元素，就可以直接留下来。

递推状态设计

逆向中，按数值从大到小处理。

设当前处理值为 x 。

令 cur 表示：

已经决定要展开的、值大于 x 的节点数量。

为什么这有用？

因为每一个被展开的更大节点，都会强制产生一个 x 。

同时，根节点 k 的展开也会强制产生一个 x 。

所以，在处理 x 时，一共有：

$$q_x = cur + 1$$

个 x 节点需要处理。

设初始集中有 c_x 个 x 。

我们最多可以留下：

$$\min(q_x, c_x)$$

个 x 节点作为最终叶子。

剩下的：

$$\max(0, q_x - c_x)$$

个 x 节点必须继续展开。

于是：

$$cur \leftarrow cur + \max(0, cur + 1 - c_x)$$

也就是：

$$cur \leftarrow \begin{cases} cur, & c_x \geq cur + 1 \\ 2cur + 1 - c_x, & c_x < cur + 1 \end{cases}$$

最后，基础答案是：

$$1 + cur$$

其中 1 是根节点 k 的展开。

为什么可以贪心？

从大到小处理时，值大的节点是否展开，会影响后面产生多少小节点。

对于当前值 x ，如果有一个初始的 x 可以直接匹配当前的 x 节点，那么一定应该匹配。

因为：

1. 匹配它的代价是 0；
2. 不匹配就必须展开它，至少多花 1 次；
3. 展开还会制造更多小值节点，只会让后续更麻烦。

所以对每个 x ，能留下多少就留下多少，这是最优的。

处理 k 本身

根节点是 k 。

根节点展开出来的集合不能包含 k 。

所以如果初始集合里有 k ，这些 k 不能直接由根节点产生。

分两种情况：

情况一：有某个小于 k 的节点被展开

也就是最终 `cur > 0`。

那么在这个被展开的小节点里，可以把所有 k 作为额外元素免费放进去。

因为如果展开的是 x ， mex 为 x 的集合禁止的是 x ，不禁止 k 。

所以不需要额外操作。

情况二：没有任何小于 k 的节点被展开

也就是 `cur == 0`。

这说明根展开一次后，所有强制产生的 $0, 1, \dots, k-1$ 都直接匹配了初始元素。

如果初始集合里还需要 k ，就必须额外制造一个非 k 的节点，再展开它来产生 k 。

因此额外加 1。

特殊地，如果初始集合本来就是 $\{k\}$ ，答案是 0。

稀疏优化

不能从 $k-1$ 枚举到 0。

如果一段值都没有出现在初始集合中，即 $c_x = 0$ ，递推为：

$$cur \leftarrow 2cur + 1$$

连续做 L 次：

$$cur \leftarrow (cur + 1)2^L - 1$$

所以只需要处理出现过的 x ，中间空段用快速幂跳过。

3. Trick 与陷阱

Trick 1: 逆向 mex

正向 mex 操作很难控制，但逆向展开非常清晰：

$$x \rightarrow \{0, 1, \dots, x-1\} + \text{任意不等于 } x \text{ 的额外元素}$$

这直接暴露了题目的树形结构。

Trick 2: `cur` 只需要维护两份

因为答案要取模，但比较时需要知道真实的 `cur` 是否超过 c_x 。

所以维护：

1. `curM`： `cur mod 998244353`；
2. `cur`：真实值的截断版，最多保留到 $n+1$ 。

因为所有 $c_x \leq n$ ，一旦 `cur > n`，之后比较时一定是 `cur + 1 > c_x`，没必要知道更大的真实值。

易错点

1. 初始集合正好是 $\{k\}$ 时，答案是 0。
2. $k = 0$ 时，没有 $0, \dots, k-1$ ，但如果初始集合里还有 0 且不是单独的 $\{0\}$ ，仍然需要额外操作。
3. 重复元素有用，但只能在对应的强制节点数量范围内有用。
4. $s_i > k$ 基本不影响代价，因为它们可以作为根展开的额外元素免费出现。
5. $s_i = k$ 需要特殊处理，因为根展开时不能直接产生 k 。

4. 代码实现 C++23

```
#include <bits/stdc++.h>

using namespace std;

using i64 = long long;

constexpr i64 P = 998244353;

i64 power(i64 a, i64 b) {
    i64 r = 1;
    while (b) {
```

```

        if (b & 1) r = r * a % P;
        a = a * a % P;
        b >>= 1;
    }
    return r;
}

int main() {
    cin.tie(nullptr)->sync_with_stdio(false);

    int T;
    cin >> T;

    while (T--) {
        int n;
        i64 k;
        cin >> n >> k;

        vector<i64> a;
        bool hask = false;

        for (int i = 0; i < n; i++) {
            i64 x;
            cin >> x;
            if (x == k) hask = true;
            if (x < k) a.push_back(x);
        }

        if (n == 1 && hask) {
            cout << 0 << "\n";
            continue;
        }

        sort(a.rbegin(), a.rend());

        i64 lim = n + 1;
        i64 cur = 0;
        i64 curM = 0;

        auto grow = [&](i64 len) {
            if (len <= 0) return;

            curM = (curM + 1) * power(2, len) % P;
            curM = (curM - 1 + P) % P;

            while (len > 0 && cur < lim) {
                cur = min(lim, cur * 2 + 1);
                len--;
            }
        };

        i64 pre = k;

```

```

    for (int i = 0; i < int(a.size()); ) {
        int j = i;
        while (j < int(a.size()) && a[j] == a[i]) j++;

        i64 x = a[i];
        i64 cnt = j - i;

        grow(pre - x - 1);

        if (cur + 1 > cnt) {
            curM = (2 * curM + 1 - cnt) % P;
            if (curM < 0) curM += P;

            if (cur < lim) {
                cur = min(lim, 2 * cur + 1 - cnt);
            }
        }

        pre = x;
        i = j;
    }

    grow(pre);

    i64 ans = (curM + 1) % P;

    if (hask && cur == 0) {
        ans = (ans + 1) % P;
    }

    cout << ans << "\n";
}

return 0;
}

```

5. 总结与启示

难度评分

预估 Codeforces Rating: **2300 左右**。

核心启示

这题的关键是：把 mex 合并操作反过来看，复杂的集合变换会变成一棵强制展开树，然后用从大到小的贪心统计必须展开的节点数。

同类习题推荐

1. Codeforces 1375D - Replace by MEX

经典 mex 构造题，训练 mex 操作的结构感。

2. AtCoder AGC032A - Limited Insertion

典型逆向思维题，和本题“反过来更简单”的思路类似。

3. Codeforces 1628A - Meximum Array

mex + 贪心分段，适合巩固 mex 的全局结构分析。